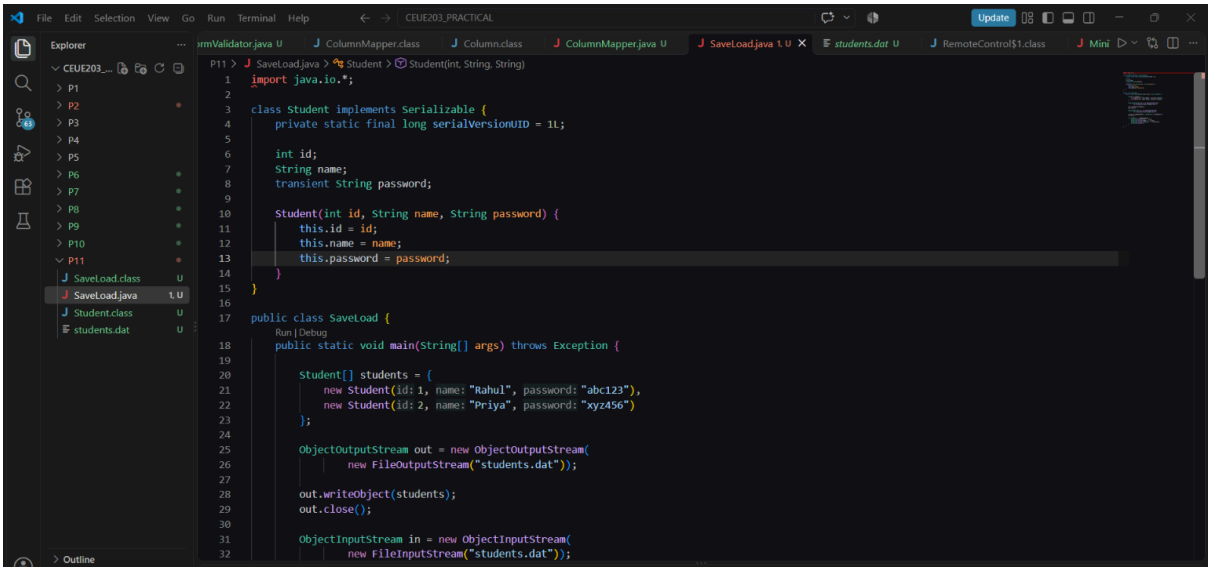
PRACTICAL 11

QUESTION 1 :

Save/load: make a small class Serializable, save an array of its objects to a file with ObjectOutputStream, then read it back into fresh objects and confirm the values survived. Mark one field transient and show it is not saved.

CODE :



```java
import java.io.*;

class Student implements Serializable {
    private static final long serialVersionUID = 1L;

    int id;
    String name;
    transient String password;

    Student(int id, String name, String password) {
        this.id = id;
        this.name = name;
        this.password = password;
    }
}

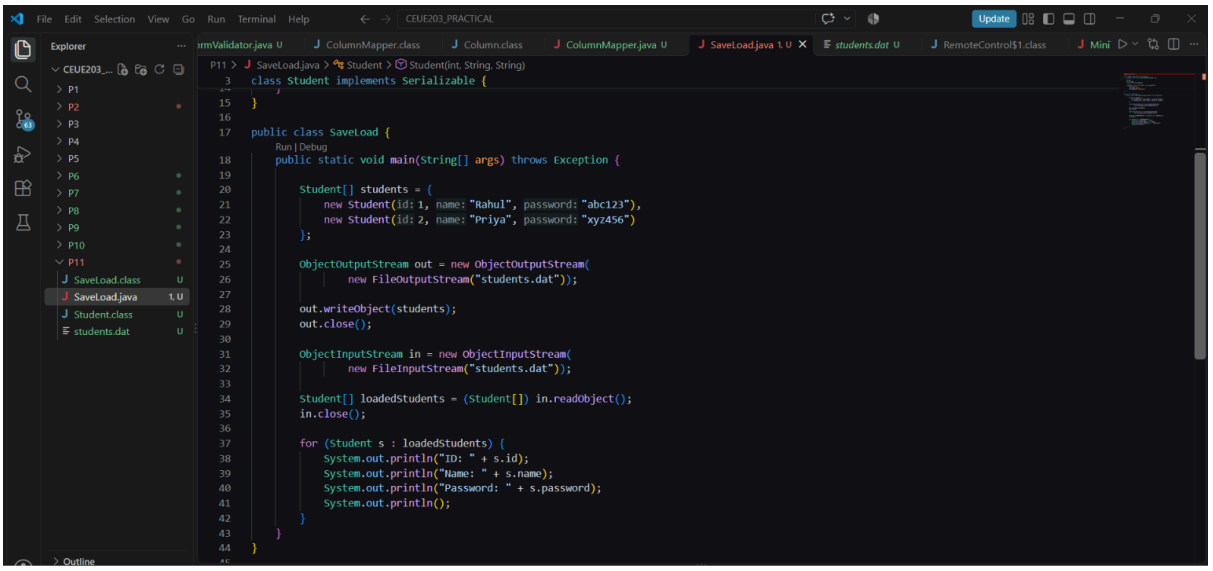
public class SaveLoad {
    Run | Debug
    public static void main(String[] args) throws Exception {

        Student[] students = {
            new Student(id: 1, name: "Rahul", password: "abc123"),
            new Student(id: 2, name: "Priya", password: "xyz456")
        };

        ObjectOutputStream out = new ObjectOutputStream(
                new FileOutputStream("students.dat"));

        out.writeObject(students);
        out.close();

        ObjectInputStream in = new ObjectInputStream(
                new FileInputStream("students.dat"));
```



```java
class Student implements Serializable {
}

public class SaveLoad {
    Run | Debug
    public static void main(String[] args) throws Exception {

        Student[] students = {
            new Student(id: 1, name: "Rahul", password: "abc123"),
            new Student(id: 2, name: "Priya", password: "xyz456")
        };

        ObjectOutputStream out = new ObjectOutputStream(
                new FileOutputStream("students.dat"));

        out.writeObject(students);
        out.close();

        ObjectInputStream in = new ObjectInputStream(
                new FileInputStream("students.dat"));

        Student[] loadedStudents = (Student[]) in.readObject();
        in.close();

        for (Student s : loadedStudents) {
            System.out.println("ID: " + s.id);
            System.out.println("Name: " + s.name);
            System.out.println("Password: " + s.password);
            System.out.println();
        }
    }
}
```

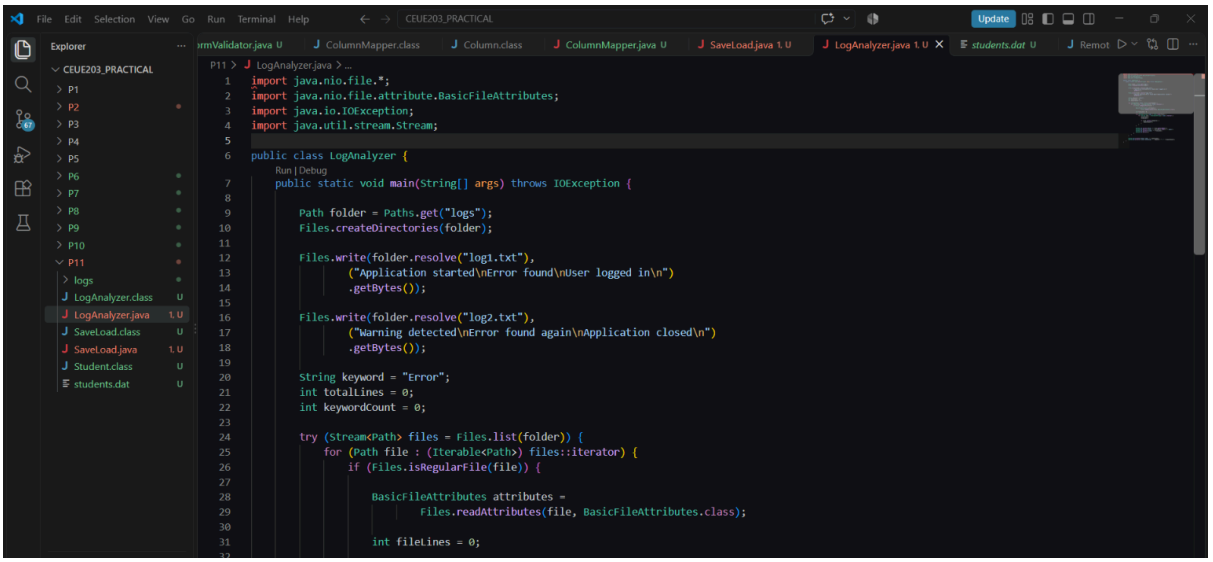
OUTPUT :

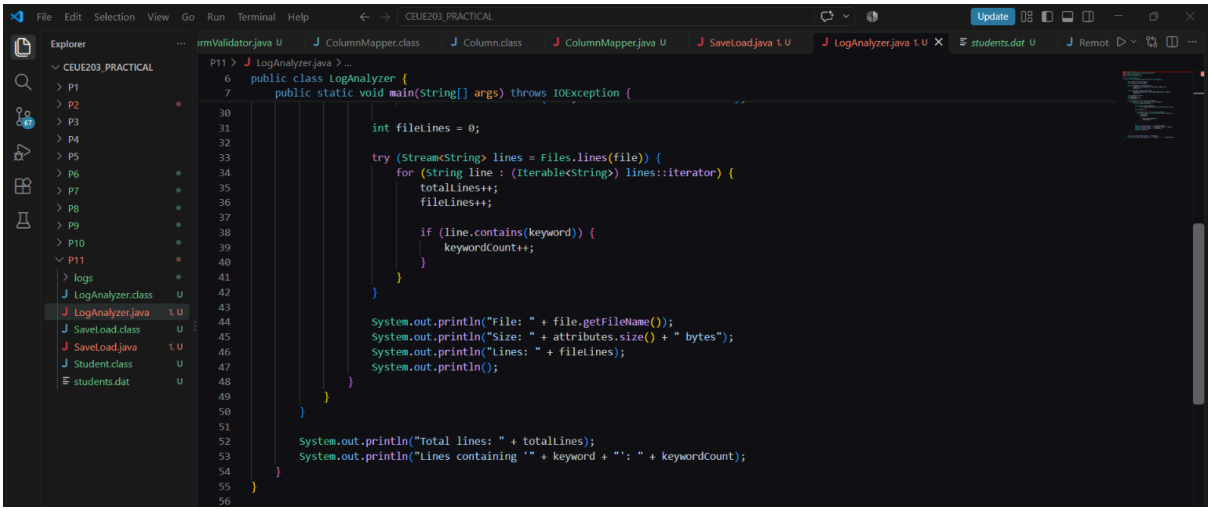


QUESTION 2

Log analyzer: create a few text files, then use Path/Files to read each line by line and count the total lines and how many contain a keyword across all files; also print each file's size from its attributes.

CODE :





```java
public class LogAnalyzer {
    public static void main(String[] args) throws IOException {

                int fileLines = 0;

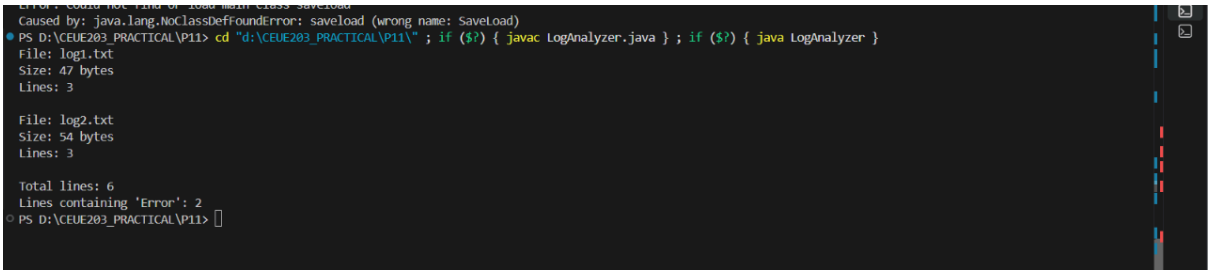
                try (Stream<String> lines = Files.lines(file)) {
                    for (String line : (Iterable<String>) lines::iterator) {
                        totalLines++;
                        fileLines++;

                        if (line.contains(keyword)) {
                            keywordCount++;
                        }
                    }
                }

                System.out.println("File: " + file.getFileName());
                System.out.println("Size: " + attributes.size() + " bytes");
                System.out.println("Lines: " + fileLines);
                System.out.println();
            }
        }
    }

    System.out.println("Total lines: " + totalLines);
    System.out.println("Lines containing '" + keyword + "': " + keywordCount);
}
}
```

OUTPUT :



```
Caused by: java.lang.NoClassDefFoundError: saveload (wrong name: SaveLoad)
PS D:\CEUE203_PRACTICAL\P11> cd "d:\CEUE203_PRACTICAL\P11\" ; if ($?) { javac LogAnalyzer.java } ; if ($?) { java LogAnalyzer }
File: log1.txt
Size: 47 bytes
Lines: 3

File: log2.txt
Size: 54 bytes
Lines: 3

Total lines: 6
Lines containing 'Error': 2
PS D:\CEUE203_PRACTICAL\P11>
```

KEY QUESTIONS :

1. What does it mean for a class to be Serializable, and why add serialVersionUID?

A class is Serializable when it implements the Serializable interface. It allows objects of that class to be saved to a file and loaded later. serialVersionUID is used to identify the version of the class during serialization and deserialization.

2. What is the difference between a byte stream and a character stream?

A byte stream reads and writes data in bytes. It is mainly used for binary data, such as images and files. A character stream reads and writes text characters. Examples are FileInputStream for byte streams and FileReader for character streams.

3. How does NIO’s Path/Files API differ from the old File class, and how is a directory stream iterated?

The old File class provides basic file and directory operations. The NIO Path and Files APIs provide more advanced features, such as reading files, accessing file attributes, and handling directories. A directory can be iterated using Files.list(path) with a stream, or Files.newDirectoryStream(path) with a directory stream.